

# Security: How We Keep Data Safe

---

**Last updated:** 2026-08-29. **Audience:** developers, IT, leadership. **Companion doc:**

`docs/DEEP_AUDIT_REPORT.md` (full technical audit)

This document explains, in plain terms, what protects employee and company data in Project-Ticket today, and what was just hardened. It reflects the state of the code as of this date. If you change login, roles, or the API surface, please update this file too.

---

---

## 1. The short version

Every piece of data in this app (complaints, HR records, assets, tasks) lives in one place, a database called Firestore, and is reachable through exactly one door: our backend server (the program that receives requests from the app and decides what to do with them). The browser never talks to the database directly. That single door is where every protection lives: who you are, what you're allowed to do, and what you're allowed to see.

Nothing here requires trusting the app you see on screen. Every check that matters, "is this really you," "are you allowed to do this," "do you own this record," is enforced again on the server, even though the on-screen app also hides buttons a role shouldn't see.

---

---

## 2. Identity: how we know who you are

- **Login** goes through Firebase Authentication (a Google service built for handling logins securely), not our own password store. We never see or hold a raw password. Firebase verifies it and hands back proof of identity. ( `main/backend/controllers/authController.js` )
- **Sessions** (the period you stay logged in) are represented by a small, signed piece of proof, kept in a browser cookie your computer holds onto automatically. It expires and quietly renews itself in the background while you're active, so you don't get logged out mid-task, and it stops working on its own well before it would ever become a lasting risk if it were somehow copied.
- **Logging out, or being logged out, actually works.** Each login creates a record of that session in the database. Logging out (by the user, or a forced logout by an admin) flips that record to "revoked" (cancelled). Every request checks this again, so a stolen or old sign-in stops working within about a minute of being cancelled, not days later.
- **Locked out after repeated wrong passwords.** Five failed attempts locks the account. Only the Founder (the platform's top-level admin account, called `superadmin` in the code) can unlock it. Every failed attempt is logged along with the internet address (IP) it came from.

- **Login attempts are rate-limited** (throttled, so a computer can't just keep guessing passwords rapidly). On top of the lockout, no more than 10 login, sign-up, or re-verify attempts are accepted from one source every 15 minutes.
  - **Passwords must be at least 10 characters.** This is enforced the same way whether you're signing up yourself, an admin is creating your account, or an admin is resetting your password. *(As of this update. Previously, signing up yourself had no real minimum length.)*
  - **You can't grant yourself a role.** Signing up always creates a plain employee account. Only the Founder can create HR, IT, coordinator, or employee accounts, through a screen that only offers those specific roles. There's no path from the signup form to picking your own permissions. Promoting an existing account to Founder-level access is a separate, even more restricted action.
- 
- 

### 3. Permissions: what you're allowed to do

Three layers, all checked on the server, not just hidden on screen:

1. **Route-level role check.** For example, only HR staff and founders can even reach the "view all HR complaints" screen's underlying request.
2. **Fine-grained permission table.** For sensitive actions like editing IT assets, a table the Founder can configure decides who can create, edit, or delete, on top of the basic role check.
3. **Ownership check.** For your own records (your complaint, your leave request), you can only edit or delete the ones that are actually yours. Trying to edit someone else's is rejected by the server even if you knew its internal ID.

**A concrete example:** if you try to delete a complaint that isn't yours, the request is refused with a permission error. The server checks who owns the record before allowing the delete, every single time, no matter what buttons the app happens to show you on screen.

---

---

## 4. What's protecting the system itself

| PROTECTION                                                                | WHAT IT DOES                                                                                                                                                                                                                                                                                                                                                  |
|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Security headers</b><br><i>(added in this update)</i>                  | A set of extra instructions ("helmet") now travels with every response, telling the browser to guard against a whole class of common web attacks by default (like tricking a browser into running malicious code, or displaying our site inside someone else's disguised page).                                                                               |
| <b>General rate limiting</b><br><i>(added in this update)</i>             | Every request to our server (not just login) is now capped at 300 per 15 minutes per source, on top of the stricter login limit. This stops a runaway script or a compromised account from hammering the database with requests.                                                                                                                              |
| <b>Trusting the real visitor address</b><br><i>(fixed in this update)</i> | We run behind a hosting provider's proxy (a relay server in front of ours). The server now correctly reads each visitor's real address from that relay instead of seeing one shared address for everyone. This is what makes the rate limits above actually apply per visitor, instead of accidentally lumping all traffic together as if it were one person. |
| <b>Allowed-website list</b>                                               | Only our own app's web address (plus a local address used during development) may call our server from a browser. Not "anyone on the internet."                                                                                                                                                                                                               |
| <b>Generic error messages</b>                                             | If something breaks unexpectedly, the person using the app sees a plain "Something went wrong" message, never a raw database error, technical crash detail, or internal file location. The real detail is only ever written to our own private server log.                                                                                                    |
| <b>File upload limits</b>                                                 | Uploaded HR documents are checked by file type (PDF, JPG, Word documents only) and capped at 10MB before they're ever saved.                                                                                                                                                                                                                                  |
| <b>Outbound email is cleaned up first</b>                                 | Complaint text going into notification emails is sanitized (stripped of anything that could be misread as a formatting or link command) first, so a complaint can't be crafted to sneak links or formatting into HR's inbox.                                                                                                                                  |

---

## 5. Where data actually lives

- **Firestore** (Google's database) holds everything: user accounts, complaints, leave requests, assets, tasks, activity logs. Nothing is duplicated into a second database.
  - **Firebase Storage** holds uploaded HR documents. Files aren't reachable just by guessing a web address. Every download goes back through our own server's permission checks first.
  - **Only our server has the keys.** The credentials that let anything talk to the database, file storage, or login system live only in the server's private settings, never in the copy of the app a browser downloads, and are kept out of the project's shared code history.
  - Your sign-in proof is held in a browser cookie your computer manages automatically, flagged so that no page script running in the browser (not even our own) can read its contents directly. That's a deliberate extra layer: even if a bug somewhere let an attacker sneak code onto one of our pages, that code still couldn't get at your sign-in proof to steal it.
-

---

## 6. What we keep a record of

- **Every admin action is recorded:** creating, editing, or deleting a user account, changing someone's permissions, resetting a password. If something goes wrong, there's a trail of who did what and when.
  - **Every failed login is recorded**, along with the internet address it came from, feeding the lockout protection described above.
  - Day-to-day request and error activity currently goes to the server's own private log rather than a separate, searchable system built for browsing history. That's good enough for the app's current size, and worth revisiting as usage grows (see section 7 below).
- 

---

## 7. Known trade-offs (nothing hidden)

We'd rather tell you what's still a work in progress than pretend everything is perfect:

- **No two-factor login yet** (a second proof of identity beyond your password, like a code sent to your phone). Account security today rests on password strength plus the lockout and rate-limit protections above.
- **No dedicated searchable log system yet.** Day-to-day request logs live in the server's own private output, not a browsable dashboard. Admin actions and failed logins are the exception; those go to the database (see section 6).
- **No automated safety check before each release yet.** Nothing currently double-checks for security or outdated-software issues automatically before new code goes live; releases go out directly from the project's code.
- A couple of internal features (updating production-render records, and an employee updating their *own* task's status) are intentionally open to any signed-in account rather than restricted by role. This was reviewed and judged low-risk given what those features actually touch, but it's worth re-checking if the app's scope grows.

None of these block using the app today. They're the honest next-hardening list, tracked in `docs/DEEP_AUDIT_REPORT.md`.

---

---

## 8. If something looks wrong

If you notice unexpected access, an account locked out that shouldn't be, or anything that looks like it shouldn't have been possible, the Founder can see and cancel active sessions, view the failed-login history, and unlock accounts from the Security section of the Founder dashboard (`securityRoutes.js`, restricted to the Founder only).